

sin r. You'd like to define a *lookup-sin* macro to do the table lookup at runtime and also hide the details of table generation, which would just clutter your program's source code.

Your macro will be invoked as (*lookup-sin radians divisions*), where *radians* is always in the range of zero to one-half π , and *divisions* is the number of discrete values available as the result of *lookup-sin*. At runtime, the macro expander will just compute the index into a lookup table, and return the value from the table. The table will be generated at compile-time (on most Lisp systems). Furthermore, only one table will ever be generated for a given value of *divisions* in the macro call.

```
;; This is where we cache all of the sine tables generated
;; during compilation.
;; The tables stay around at runtime so they can be used for
lookups.
(defvar *sin-tables* (make-hash-table))
    "A has table of tables of sine values. The hash is keyed
by the number of entries in each sine table.")

;; This is a helper function for the lookup-sin macro. It is
used only at compile
;; time.
(defun get-sin-table-and-increment (divisions)
    "Returns a sine lookup table and the number of radians
quantised by each entry in the table. Tables of a given size
are reused. A table covers angles from zero to pi/2 radians."
  (let ((table (gethash divisions *sin-tables* :none))
        (increment (/ pi 2 divisions)))
    (when (eq table :none)
      ;; Uncomment the next line to see when a table gets
      created.

      ;; (print "Making new table")
      (setf table
        (setf (gethash divisions *sin-tables*)
          (make-array
            (1+ divisions)
            :initial-element 1.0)))
        (dotimes (i divisions)
          (setf (aref table i)
            (sin (* increment i)))))
        (values table increment)))

;; Macro calls the helper at compile time, and returns an
AREF form to do the
;; lookup at runtime.
(defmacro lookup-sin (radians divisions)
    "Return a sine value via table lookup."
  (multiple-value-bind (table increment)
    (get-sin-table-and-increment
```

```
divisions)
  `(aref, table (round, radians, increment))))
```

Let us examine what is happening here. When this program runs, it executes just *aref* (and associated *round*) to look up the *sin r* value.

```
> (pprint (macroexpand-1 `(lookup-sin (/ pi 4) 50)))

(AREF #(0.000 0.0314107590781282900 0.0627905195293133800
0.0941083133185143300 0.1253332335643042600
0.1564344650402308700 0.1873813145857246300
0.2181432413965425700 0.248689887164854800 0.2789911066039229300
0.309016994374947400 0.338737928245291400
0.36812455268467800 0.397147890634780600 0.425779291565072700
0.453990499739546800 0.481753674181715300 0.509041415750371300
0.535826794978996700 0.562083377852130600
0.587785252292473100 0.612907853652976500
0.637423989748689800 0.661311865323651800 0.684547105928688700
0.707106781186547600 0.728986627421411600 0.750111069630459600
0.770513242775789300 0.790155012375690400
0.809016994374947500 0.827808574274561800
0.844327925502015100 0.860742627803943000 0.876306680043863700
0.891006524188367800 0.904827052466019000 0.917754625683981100
0.929776485808251500 0.940808768954225600
...)
(ROUND (/ PI 4) 0.03141592653589793400))
```

Note that the macro call makes no mention of a lookup table. Tables are generated as needed by (and for) the compiler.

```
> (lookup-sin (/ pi 4) 50)
0.707106781186547600
```

In the macro expansion, the *#(...)* is the printed representation of the lookup table for 50 divisions of the quarter circle. This table is stored in the **sin-tables** hash table, where it is shared by every macro call to (*lookup-sin angle 50*). We don't even have to do a hash lookup at runtime, because the macro expander has captured the free variable table from the multiple-value-bind form in *lookup-sin*.

Macros that define macros

Macros that define macros are used infrequently, partly because it's hard to think of a good use for this technique, and partly because it's difficult to get right. The following macro, based on an example in Paul Graham's book 'On Lisp', can be used to define synonyms for the names of Lisp functions, macros, and special forms.

```
> (defmacro defsynonym (old-name new-name)
    "Define OLD-NAME to be equivalent to NEW-NAME when used in the
```